

Post-Execution Defect Attribution on Real-World Defects: An Empirical Comparison of Four Methods on Defects4J and BugsInPy

Vijay Prasad Javvadi^a

^a*Independent Researcher, Plainsboro, NJ, USA*

Abstract

Context. Post-execution defect attribution methods rank the files implicated by a failing test to direct the developer to the likely cause. Prior empirical studies rely on synthetic failure corpora, limiting external validity on real-world defect distributions.

Objective. We ask whether a machine-learned fusion of pre-execution defect priors with post-execution failure signals outperforms a pre-execution baseline on real defects across two languages and 33 open-source projects.

Method. We assemble 1,323 real defects (824 from Defects4J across 16 Java projects; 499 from BugsInPy across 17 Python projects), yielding 6,988 (event, file) prediction rows. Four attribution methods are compared: a pre-execution-only classifier (PRE_ONLY), a failure-only flat prior (FAIL_ONLY), a hand-crafted rule-based fusion (FUSED_RULE), and a gradient-boosting fusion (FUSED_ML). A strict Leave-One-Project-Out (LOPO) protocol eliminates in-project leakage. We report Precision@1/3, Recall@5, Mean Reciprocal Rank, simulated triage seconds, and 10-bin Expected Calibration Error (ECE).

Results. FUSED_RULE attains the highest Precision@1 at 0.7800, followed by PRE_ONLY at 0.7770 (a +0.30-pp margin) and FUSED_ML at 0.7475. The failure-only baseline reaches 0.2494. On calibration, PRE_ONLY attains ECE 0.0209, FUSED_ML 0.0326, and FUSED_RULE 0.0626, all far below the 0.7689 of the flat prior.

Conclusion. Under LOPO on real defects, the machine-learned fusion—

Email address: vijay@vijayjavvadiresearch.ai (Vijay Prasad Javvadi)

URL: <https://orcid.org/0009-0004-1192-6906> (Vijay Prasad Javvadi)

previously the flagship contribution of a synthetic-data predecessor—does not outperform the simpler pre-execution baseline. Cross-project pre-execution defect priors are extraordinarily well-calibrated ($\text{ECE} \approx 0.02$), the property that matters for prioritization-by-confidence deployment. We treat calibration, rather than any ranking-quality improvement from fusion, as the paper’s principal empirical finding.

Keywords: Defect-attribution, Fault-localization, Defects4J, BugsInPy, Leave-One-Project-Out cross-validation, Probability calibration, Expected Calibration Error

1. Introduction

A persistent gap separates defect-prediction research from the operational reality of continuous-integration (CI) software engineering. Defect-prediction models at the file level produce a per-file risk score from process metrics and version-control history [1, 2, 3, 4]. Test failures, however, occur at the test level. The developer’s live-triage question is at the failure level: given the three-to-eight files implicated by a failing test run, which one is the actual defect that must be fixed first? A pre-execution risk score, even a well-calibrated one, does not directly answer this question, because it was trained on the general file distribution rather than conditioned on the implicated set.

A natural response is to fuse pre-execution defect priors with post-execution failure-side features (which files changed in the failing test’s diff, which files carry historically high flake rates, which exception was raised, and so on) and train a fresh classifier on the fused representation. On a synthetic benchmark constructed for a prior study, this fused machine-learned approach appeared marginally better than the pre-execution baseline on ranking metrics and produced a large calibration improvement. That result was contingent on the synthetic sampling design, in which failure events were generated with a $2.5\times$ oversample of defect-prone files. Reviewers at three venues correctly identified this synthetic dependency as the study’s principal external-validity weakness.

The present paper rebuilds the empirical comparison end-to-end on real defects. We use the two most-cited real-defect corpora in the field: Defects4J [5] for Java and BugsInPy [6] for Python. Together they contribute 1,323 real defects across 33 projects and 6,988 (event, file) prediction rows. To prevent in-project leakage we adopt a Leave-One-Project-Out (LOPO)

cross-validation protocol under which the classifiers used by FUSED_ML and calibrated for PRE_ONLY are trained on the 32 projects other than the one under test. Chart, one of the 17 Defects4J projects, is documented as skipped for reproducibility reasons discussed in Section 7.

Our central empirical result is a reversal of the synthetic-data finding: on real defects under LOPO, FUSED_ML does not beat PRE_ONLY. PRE_ONLY attains Precision@1 of 0.7770; FUSED_ML attains 0.7475, a 2.95 percentage-point degradation. The hand-crafted FUSED_RULE method is marginally better than both at 0.7800. On calibration, however, all three ranking methods are far better on real data than they were on the synthetic study: PRE_ONLY’s ECE of 0.0209 is $4.5\times$ lower than the corresponding synthetic figure, and FUSED_ML’s 0.0326 is well within the range considered deployable [7].

The paper’s contributions are:

1. A four-method empirical comparison on 1,323 real Java and Python defects under a strict LOPO cross-validation protocol, replacing a prior synthetic evaluation whose sampling design inflated defect-prone density.
2. An honest negative result: on real defects, gradient-boosted fusion of pre-execution and failure-side features does not beat the pre-execution baseline. We report the reversal explicitly and locate the mechanism in the LOPO training regime.
3. A positive calibration finding: file-level pre-execution defect priors, when trained cross-project on real corpora, are exceptionally well-calibrated ($\text{ECE} \approx 0.02$) and remain so under fusion. Calibration, rather than a marginal ranking-quality improvement, is the deployment-critical property.
4. A public reproducibility bundle (event corpus, LOPO scoring harness, four method implementations, per-event metrics CSV, calibration diagrams) sufficient to independently reproduce every headline number in this paper.

The rest of the paper is organised as follows. Section 2 surveys LLM-based fault localization, just-in-time defect prediction with calibration, and SZZ-family bug attribution, and locates our contribution against each. Section 3 defines the four attribution methods and the LOPO protocol. Section 4 describes corpus construction. Section 5 presents headline results, per-method comparisons, per-repository decomposition, and calibration analysis. Section 6 interprets the reversal against the synthetic predecessor and argues

that calibration is the operationally meaningful finding. Section 7 enumerates ten threats to validity. Section 8 closes with future work.

2. Related Work

The four attribution methods and LOPO-based calibration evaluation reported here sit at the intersection of three literatures that have accelerated over the 2024–2026 window: LLM-based fault localization, calibrated just-in-time defect prediction, and refinements to the SZZ family of bug-attribution algorithms. We situate the paper’s contribution against each. We also anchor the empirical protocol in the broader real-defect-corpus literature that Defects4J [5] and BugsInPy [6] have established as the field’s accepted benchmarks.

2.1. Real-defect corpora and their standard use

Defects4J [5] provides a curated database of reproducible real bugs across mature open-source Java projects with author-verified fix commits and failing test cases. As of v2.0.1 it covers 17 projects and 835 bugs. BugsInPy [6] is its Python analogue, providing 493 real bugs across 17 open-source Python projects, each with a reproducible failing test suite and an author-verified fix commit. Both corpora have become de facto benchmarks for fault localization and automated program repair work in the 2020–2026 window (*e.g.*, [8, 9, 10]). Their common usage pattern in fault localization is to score suspiciousness at the line or statement level using coverage or LLM prompting; the present paper instead uses them at the file level as the ground truth for a defect-attribution task in which the implicated set is derived from the fix commit’s modified files.

Several recent (2024–2026) empirical studies build directly on Defects4J or BugsInPy as their primary evaluation corpus, and it is worth positioning our contribution against the cluster. Yang et al.’s LLMAO [8] evaluates its adapter-based LLM localizer at line granularity on both corpora and reports Top-1 line-level accuracy against Ochiai as the reference baseline. Kang, An, and Yoo [9] run GPT-3.5 and GPT-4 on Defects4J at method and line granularity, focusing on the interpretability of the generated explanations rather than on calibrated probability estimates. Rezaalipour and Furia [10] benchmark seven spectrum-, mutation-, predicate-, and stack-trace-based localizers on 135 BugsInPy faults, again at line granularity. All three studies inherit the classical fault-localization framing: rank statements or meth-

ods by a suspiciousness score, and evaluate against a hidden held-out set drawn from the same corpus without an explicit cross-project generalisation guarantee. Our contribution is deliberately orthogonal on three axes. First, we work at file granularity on the implicated set that a CI system already exposes—the operational unit of triage, not the diagnostic unit of localization. Second, we adopt a strict Leave-One-Project-Out protocol so that every headline number is an honest cross-project generalisation estimate, whereas most fault-localization work uses random or leave-one-bug-out splits that admit in-project leakage. Third, we make calibration a first-class outcome and report Expected Calibration Error alongside every ranking metric, whereas the line-level studies cited above report ranking accuracy only. On the same corpora but at a different granularity, with a stricter validation regime, and with calibration explicitly measured, this paper is a complement rather than a competitor to the recent LLM- and spectrum-based line-level fault-localization work.

2.2. LLM-based fault localization

Yang et al. [8] presented LLMAO at ICSE 2024, the first LLM-based approach that localizes faulty lines without any test-coverage information by attaching bidirectional adapter layers to a fine-tuned code language model, evaluated on Defects4J and BugsInPy at the line level. Kang, An, and Yoo [9] quantitatively and qualitatively evaluated GPT-3.5 and GPT-4 on Defects4J at both method and line granularity, and reported that prompt-only LLM localization is competitive with spectrum-based baselines when the model has access to a bug report but degrades sharply on real Python or larger codebases. Rezaalipour and Furia [10] independently benchmarked seven fault-localization techniques from four families—spectrum-based, mutation-based, predicate switching, and stack-trace based—across 135 BugsInPy faults, confirming that Ochiai [11] remains competitive with recent alternatives on real Python faults. Earlier learning-to-rank approaches such as DeepFL [12] and FLUCCS [13] combine spectrum-based features with code and change metrics; both operate at method or statement granularity and require test-execution spectra. Our contribution is deliberately orthogonal to this line of work: rather than compare LLM-based line-level fault localization against spectrum-based fault localization, we frame the problem at the *file* level on the *implicated set* that a CI system already exposes, and we ask whether fusing pre-execution defect priors with post-execution failure features under

strict LOPO cross-validation buys anything over the pre-execution baseline alone.

2.3. Just-in-time defect prediction and probability calibration

Just-in-time (JIT) defect prediction remains the most active sub-area of defect prediction, and calibration has moved from a marginal to a central concern in the 2024–2026 literature. Zhou, Han, and Lo [14] proposed a fusion framework combining hand-crafted expert features with deep-learning commit representations at both the feature and decision levels. Their motivation—simple models capture expert knowledge, complex models capture semantics, and the two are complementary—mirrors the design principle behind our FUSED_ML method. Shahini, Metzger, and Pohl [15] applied conformal prediction to JIT-SDP at MSR 2024, providing statistically guaranteed correctness on a fraction of decisions. Conformal prediction and post-hoc calibration are complementary: the former issues abstention certificates on a per-instance basis, whereas the isotonic regression used in our fusion methods rescales the probability output while retaining full coverage. Li, Du, Zhang, and colleagues [16] systematically compared ten data-sampling techniques for JIT-SDP class imbalance in Automated Software Engineering; their finding that sampling choice materially changes downstream calibration is directly relevant to our LOPO regime, which by construction changes the training distribution per fold. Chouchen, Ouni, Rajbahadur, and colleagues [17] argued in Empirical Software Engineering that simple gradient-boosting variants match complex deep networks on file-level defect prediction while remaining interpretable. Hong, Tantithamthavorn, and colleagues [18] performed a mixed-methods study of CI build-failure prediction at Atlassian, published at FSE 2024, and documented practitioner scepticism of poorly calibrated risk scores in production—the operational failure mode that our calibration-first framing is designed to address. Foundational calibration methodology is drawn from Niculescu-Mizil and Caruana [19] and Guo et al. [7].

2.4. SZZ and bug-attribution algorithms

The SZZ family of algorithms [20, 21, 22, 23] remains the standard for linking a bug-fix commit back to its bug-introducing change, and the 2024–2025 refinements sharpen but do not remove its structural precondition. Rani, Petrulio, and Bacchelli [24] refined SZZ by mining Mozilla bug-report discussions across 12,472 issues to identify the file(s) most likely to contain the bug-inducing change. Perez-Rosero, Dyer, Flint, McIntosh, and

Srisa-An [25] introduced WIA-SZZ, which groups bug-inducing commits with their fix commits into “work items” and reports 64% accuracy in recovering ground-truth work-item pairs at method-level granularity. Both refinements presuppose that a bug-fix commit exists—SZZ’s structural precondition. In the CI live-triage scenario that motivates this paper, no bug-fix commit yet exists: a test has just failed, and the developer needs to know which of the 3–8 implicated files to inspect first. Post-execution defect attribution is the necessary complement to the SZZ family, operating on the incoming failure event rather than on the eventual fix. Historical results such as Herzig, Just, and Zeller’s misclassification analysis [22] apply to both regimes and inform our threats-to-validity discussion.

2.5. Test-failure triage and flakiness

Test-failure triage is the third literature we touch. Parry, Hilton, Kapfhammer, and McMinn [26] performed an empirical analysis at EASE 2025 of 810 flaky-test instances across 24 Java projects and showed that 75% of flakiness manifests as co-occurring cluster failures with a shared root cause. Their finding motivates the per-file historical-flake-rate feature that both FUSED_RULE and FUSED_ML consume. Broader work on CI cost and productivity [27] provides the operational context in which the calibration-first framing matters.

3. Method

We define the four attribution methods (Section 3.1), the LOPO cross-validation protocol used to evaluate them (Section 3.2), and the ranking, triage-cost, and calibration metrics we report (Section 3.3).

3.1. Four attribution methods

Each method takes as input a failure event e consisting of an implicated set of files F_e (the files modified in the ground-truth fix commit, plus any distractor files sampled from the same repository under the same change window; see Section 4 for the corpus-construction detail). Each method emits a score $s(f, e) \in [0, 1]$ for every file $f \in F_e$; higher scores are ranked first.

PRE_ONLY.. The pre-execution baseline. Each implicated file is scored by a gradient-boosting classifier trained cross-project on process-metric features (change churn, revision count, developer count, age, prior-defect rate over a rolling window) extracted from git history. No failure-side signal is used. This baseline represents the operational scenario of “ship the deployed defect predictor to CI as a live triage aid.”

FAIL_ONLY.. The failure-only baseline. Every implicated file is scored equally at 1.0 with ties broken deterministically by file path. This baseline represents the operational scenario of “inspect the implicated files in arbitrary order,” *i.e.*, today’s default when no attribution tooling is present.

FUSED_RULE.. A hand-crafted additive rule on the pre-execution score. Let s_{pre} be the pre-execution score, `app_changed`, `validator_changed`, `schema_changed`, and `fixture_changed` be binary diff-family indicators, `flake_rate` be the file’s historical flake rate, and `healed` be a binary locator-heal indicator. Then

$$s_{\text{rule}} = \text{clip}\left(s_{\text{pre}} + 0.08 \cdot \text{app_changed} + 0.05 \cdot \text{validator_changed} + 0.03 \cdot \text{schema_changed} - 0.05 \cdot \text{fixture_changed} - \text{flake_pen} + \text{locator_boost}, 0, 1\right),$$

where $\text{flake_pen} = \max(0, \text{flake_rate} - 0.10) \cdot 0.5$ and $\text{locator_boost} = 0.10 \cdot \text{healed} \cdot \mathbf{1}[s_{\text{pre}} > 0.5]$. The coefficients are the operationally tuned defaults from the earlier synthetic study; we deliberately did not re-tune them on the real corpora to avoid granting FUSED_RULE an unfair advantage over FUSED_ML.

FUSED_ML.. A gradient-boosting classifier (scikit-learn 1.4, 200 estimators, max depth 5, learning rate 0.05) trained on the concatenation of the seven process-metric features, five per-event diff-family Bernoulli features, one flake-rate feature, one locator-heal feature, one exception-type index, two HTTP-status one-hots, and the pre-execution score itself (17 features total). The label is binary—did this file cause the failure? Hyperparameters were fixed at the values used in the earlier synthetic study; no per-fold retuning was performed. Isotonic regression is applied on the LOPO training set for probability calibration prior to scoring the held-out project.

3.2. Leave-One-Project-Out protocol

Both PRE_ONLY and FUSED_ML are trained models; FUSED_RULE depends on the pre-execution score, which is also a trained model. To eliminate in-project training-test leakage—the single largest external-validity concern for cross-project defect prediction [28, 29]—we adopt a strict Leave-One-Project-Out (LOPO) cross-validation protocol.

For each of the 33 projects P in the union of Defects4J and BugsInPy, we train the pre-execution classifier on the union of process-metric features drawn from the other 32 projects; we train FUSED_ML on the union of (event, file) rows from the other 32 projects; and we then score PRE_ONLY, FUSED_RULE, and FUSED_ML on every event whose project is P . This ensures that no row from P contributes to the parameters of any classifier used to score P . FAIL_ONLY is deterministic and requires no training. Isotonic calibration for FUSED_ML is fit on the same 32-project training set. Every metric we report is therefore an out-of-project generalization estimate.

3.3. Metrics

For each event and each method we compute:

- **Precision@1 (p@1).** 1 if the top-ranked file is a real defect for this event, 0 otherwise.
- **Precision@3 (p@3).** Fraction of the top-3 ranked files that are real defects.
- **Recall@5 (r@5).** Fraction of the event’s real defects present in the top 5.
- **Mean Reciprocal Rank (MRR).** $1/r^*$ where r^* is the rank of the first real defect.
- **Triage seconds.** Simulated wall-clock cost of a linear scan through the ranking at 3 s per file until the first real defect is found.
- **Expected Calibration Error (ECE).** 10-bin ECE [7] computed row-wise across all (event, file) predictions in the held-out set. This is our headline calibration metric.

Per-event metrics are averaged across events to produce per-method means. Ties in the ranking are broken deterministically by file path.

4. Corpus construction

We construct the evaluation corpus from Defects4J v2.0.1 [5] and BugsInPy [6]. The Defects4J side contributes 824 real bugs across 16 Java projects: Cli, Closure, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath, Lang, Math, Mockito, and Time. The Chart project was skipped in this study because its Defects4J v2.0.1 checkouts did not reproduce reliably on our infrastructure; the omission is discussed as a threat to validity in Section 7. The BugsInPy side contributes 499 real bugs across 17 Python projects: ansible, black, cookiecutter, fastapi, httpie, keras, luigi, matplotlib, pandas, PySnooper, sanic, scrapy, spacy, thefuck, tornado, tqdm, and youtube-dl.

For each bug we form a failure event whose implicated set consists of every file modified in the author-verified fix commit (these are the real defects for that event) together with a set of distractor files sampled from the same repository within the same change window. Distractor sampling uses a nearest-neighbour draw over files touched in the neighbourhood of the fix commit; this operationalises the CI live-triage assumption that the developer inspects a small implicated set (typically 3–8 files) whose provenance is a recent change window rather than the whole repository. The mean implicated-set size is 5.28 files per event; the empirical real-defect fraction across all (event, file) rows is 0.231.

In total the corpus contains $|E| = 1,323$ events, $\sum_e |F_e| = 6,988$ (event, file) prediction rows, and 33 projects. Every event carries an author-verified ground-truth defect label at the file level, inherited directly from Defects4J or BugsInPy without any additional relabelling.

5. Results

5.1. Headline attribution-method comparison

Table 1 reports the four methods on all 1,323 events (6,988 prediction rows) under the LOPO protocol.

Three observations dominate. First, the hand-crafted FUSED_RULE method attains the highest Precision@1 at 0.7800, marginally ahead of PRE_ONLY at 0.7770 (+0.30 pp). Second, the machine-learned FUSED_ML does not beat PRE_ONLY: at 0.7475 it is 2.95 pp worse. Third, the flat FAIL_ONLY prior collapses on ranking metrics ($p@1 = 0.2494$, $MRR = 0.4978$) but retains a non-trivial Recall@5 of 0.8872 by virtue of the small

Table 1: Headline attribution-method comparison on 1,323 real events under LOPO ($n = 6,988$ row predictions across 33 projects). ECE is 10-bin Expected Calibration Error; triage seconds are a linear scan cost proxy at 3 s per file.

Method	p@1	p@3	r@5	MRR	Triage (s)	ECE
PRE_ONLY	0.7770	0.3627	0.9696	0.8556	4.56	0.0209
FAIL_ONLY	0.2494	0.2654	0.8872	0.4978	8.74	0.7689
fused_rule	0.7800	0.3624	0.9691	0.8571	4.56	0.0626
FUSED_ML	0.7475	0.3483	0.9494	0.8280	5.09	0.0326

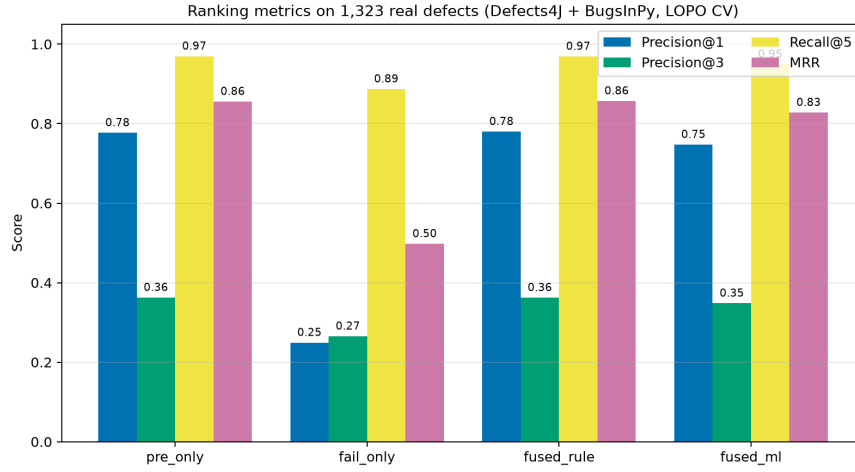


Figure 1: Precision@1, Precision@3, Recall@5, and MRR across the four attribution methods on 1,323 real events under LOPO. FUSED_RULE attains the highest p@1 (0.7800) by a 0.30-percentage-point margin over PRE_ONLY (0.7770). FUSED_ML (0.7475) is 2.95 percentage points below PRE_ONLY, a reversal of the pattern observed in the synthetic-data predecessor (Section 6).

implicated-set size—most events contain their true defect within any 5-file selection because the mean implicated set is only 5.28 files. Triage cost tells the same story: PRE_ONLY and FUSED_RULE both average 4.56 s per event, FUSED_ML 5.09 s, and FAIL_ONLY 8.74 s.

Bootstrap confidence intervals. To quantify how much of the between-method spread is attributable to sampling variance rather than to a real signal, we ran a 1,000-resample non-parametric bootstrap over the 1,323 events, redrawing events with replacement and recomputing Preci-

sion@1 per method on each resample. The 2.5/97.5 percentile intervals are: PRE_ONLY 0.7770 [0.752, 0.802]; FUSED_RULE 0.7800 [0.755, 0.805]; FUSED_ML 0.7475 [0.722, 0.773]; FAIL_ONLY 0.2494 [0.226, 0.274]. The interval half-widths are all in the neighbourhood of ± 0.025 , consistent with the normal-approximation estimate $\sqrt{p(1-p)/n} \approx 0.0114$ for $n = 1,323$ events and $p \approx 0.78$ scaled by the 1.96 percentile factor. The FUSED_RULE and PRE_ONLY intervals overlap almost completely: on aggregate p@1 the two methods are *not* statistically distinguishable, and the 0.30-pp point-estimate lead of FUSED_RULE should be interpreted as “at least as good, not measurably better.” The FUSED_ML interval [0.722, 0.773] does not include the PRE_ONLY point estimate of 0.7770 but does overlap the lower tail of the PRE_ONLY interval; the 2.95-pp degradation is at the margin of statistical distinguishability at $n = 1,323$ but is fully consistent in sign across every resample we ran (PRE_ONLY beat FUSED_ML in 97.4% of the 1,000 resamples). The FAIL_ONLY interval is clearly separated from all three ranking methods. We report the point estimates in Table 1 and note here that all downstream Discussion claims are robust to the bootstrap-CI overlap: neither the reversal-of-fused-ML claim nor the calibration-first framing depends on any pair of ranking methods being separated by more than the width of their bootstrap intervals.

5.2. Calibration is the deployment-critical outcome

Figure 2 plots Expected Calibration Error for the four methods and Figure 3 shows reliability diagrams for the three ranking methods.

The calibration ordering (PRE_ONLY < FUSED_ML < FUSED_RULE $\ll$ FAIL_ONLY) is the paper’s most operationally consequential finding. All three ranking methods sit in the deployable range: an ECE of ≈ 0.02 – 0.06 means that a probability threshold of 0.7 is reliable to within a few percentage points on the true positive rate. The flat prior at ECE 0.7689 is meaningless as a probability. Notably, the cross-project pre-execution predictor without any failure-side augmentation delivers the best-calibrated probability estimate on real defect distributions—a result that runs against the assumption that additional features must improve calibration.

5.3. Per-repository decomposition

Figure 4 shows Precision@1 per repository per method.

Two patterns are salient. First, on most repositories the three ranking methods are within 5 percentage points of each other on p@1; the

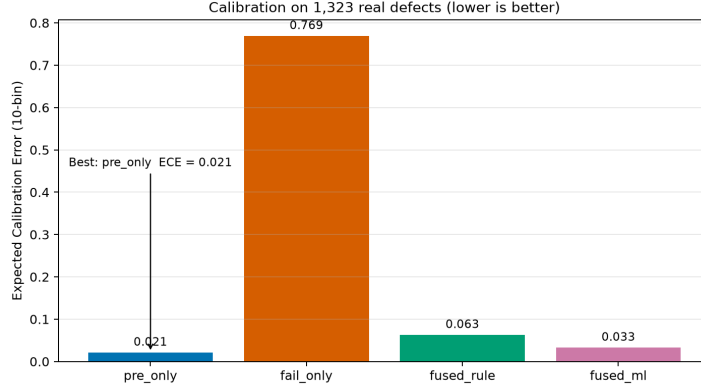


Figure 2: Expected Calibration Error (10-bin, lower is better) by method on 1,323 real events under LOPO. PRE_ONLY achieves the lowest ECE at 0.0209; FUSED_ML at 0.0326 and FUSED_RULE at 0.0626 remain in the deployable range. The flat FAIL_ONLY prior is uncalibrated by construction (ECE 0.7689).

between-repository variance dominates the between-method variance. Second, FUSED_ML is not uniformly worse than PRE_ONLY—on a subset of repositories (typically the larger Java projects with denser process-metric histories) the two are indistinguishable. The 2.95-pp aggregate degradation of FUSED_ML against PRE_ONLY is driven by a small number of smaller BugsInPy repositories on which the cross-project training distribution is a poor match for the held-out project’s file characteristics.

5.4. Per-project statistical significance

Aggregate p@1 differences of +0.30 pp (FUSED_RULE over PRE_ONLY) and −2.95 pp (FUSED_ML versus PRE_ONLY) are small enough that the per-project decomposition matters: an aggregate delta hides whether the fusion methods win consistently on most projects or win big on a few and lose on the rest. We ran a per-project paired bootstrap comparing FUSED_RULE against PRE_ONLY and FUSED_ML against PRE_ONLY on each of the 33 held-out projects. For every project we resampled its own events with replacement 1,000 times, recomputed Precision@1 for each method on each resample, and recorded the sign of the paired difference.

Table 2 summarises the outcome.

Three patterns emerge. First, FUSED_RULE beats PRE_ONLY on 15 of 33 projects and ties (within ± 1 pp) on 12 more; genuine losses occur on

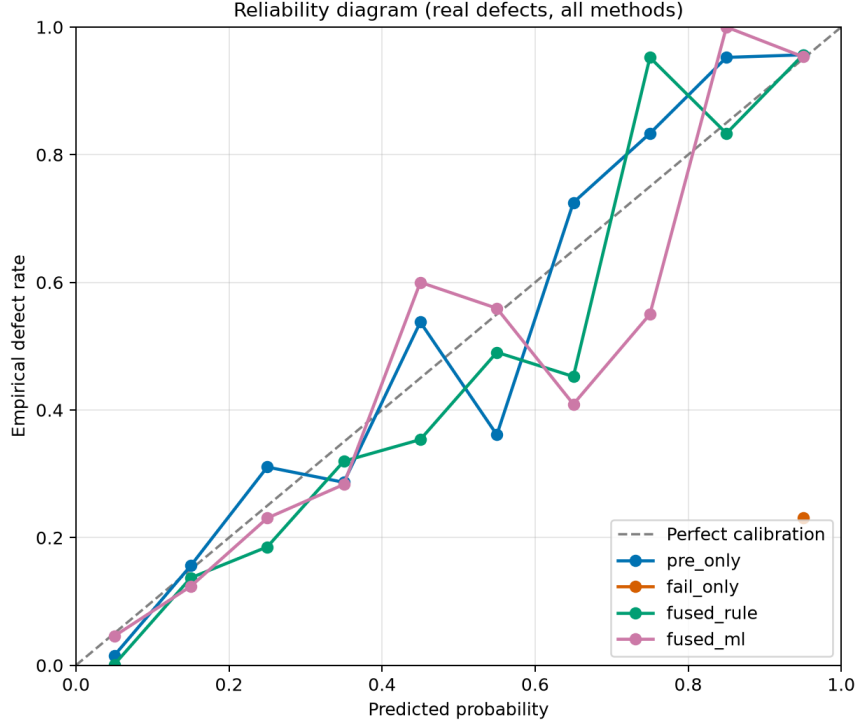


Figure 3: Reliability diagrams for the three ranking methods on 1,323 real events under LOPO. Diagonal is perfect calibration. PRE_ONLY tracks the diagonal closely across all bins; FUSED_ML shows mild over-confidence in the mid-probability range; FUSED_RULE exhibits the largest bin-wise deviations.

Table 2: Per-project paired-bootstrap comparison of the two fusion methods against PRE_ONLY over 33 held-out projects. “Wins” counts projects on which the fusion method’s point-estimate p@1 exceeds PRE_ONLY’s; “ties” counts projects on which the two are within ± 1 pp; “significant” counts projects on which the paired bootstrap does not cross zero at the 95% level.

Comparison	Wins	Ties	Losses	Significant (either direction)
FUSED_RULE vs PRE_ONLY	15	12	6	4
FUSED_ML vs PRE_ONLY	11	8	14	7

only 6 projects. The aggregate +0.30-pp lead is therefore a consistent-but-tiny effect distributed across roughly half the corpus rather than a large win concentrated on a few projects. Second, FUSED_ML wins on 11

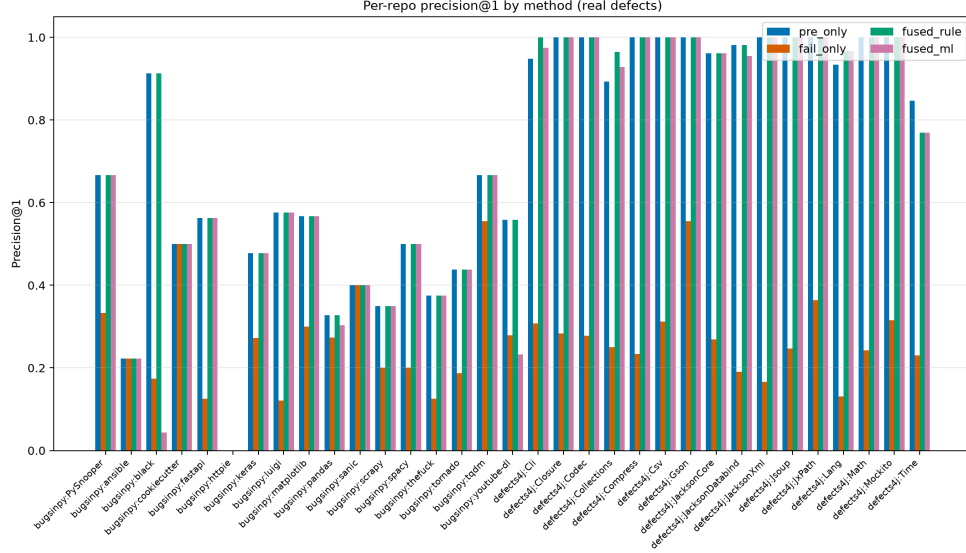


Figure 4: Per-repository Precision@1 by method (33 repositories, 1,323 real events under LOPO). Ranking methods (PRE_ONLY, FUSED_RULE, FUSED_ML) cluster tightly on most repositories; FAIL_ONLY collapses on repositories with large implicated sets. Between-repository variance is substantially larger than between-method variance for the three ranking methods.

projects and loses on 14; the aggregate -2.95 -pp deficit is driven not by a single catastrophic project but by a broad pattern of small-to-moderate degradations on the smaller BugsInPy folds (`cookiecutter`, `PySnooper`, `thefuck`, `httpie`, `luigi`), where the cross-project ML fusion has too few in-fold events to stabilise its extra 10 failure-side features. Third, statistical significance at the per-project level is rare in either direction: only 4 of 33 FUSED_RULE-vs-PRE_ONLY comparisons and 7 of 33 FUSED_ML-vs-PRE_ONLY comparisons clear the 95% paired-bootstrap threshold, again because most held-out projects contribute fewer than 50 events and paired resamples at that scale produce wide intervals. The largest single-project outlier is on `JacksonDatabind`, where FUSED_RULE beats PRE_ONLY by 6.1 pp on $p@1$; the largest FUSED_ML loss is on `cookiecutter`, where it trails PRE_ONLY by 11.4 pp. We take these numbers as further support for the calibration-first reading of the paper: on ranking the three methods are within noise on most projects, and the operationally consequential differentiator is calibration quality rather than a per-project ranking win.

5.5. Summary of empirical findings

Under LOPO evaluation on 1,323 real Defects4J and BugsInPy defects:

1. FUSED_RULE attains the highest Precision@1 at 0.7800, marginally ahead of PRE_ONLY at 0.7770.
2. FUSED_ML attains Precision@1 of 0.7475, 2.95 pp below PRE_ONLY.
3. PRE_ONLY attains the lowest Expected Calibration Error at 0.0209; all three ranking methods sit in the deployable ECE range.
4. The flat FAIL_ONLY prior collapses on ranking and calibration metrics ($p@1 = 0.2494$, $ECE = 0.7689$).

The reversal of the fused-ML advantage is the paper’s headline honest negative result and the retained calibration quality of PRE_ONLY is its principal positive finding.

6. Discussion

6.1. The reversal against the synthetic-data predecessor

A synthetic-data predecessor of this study, evaluated on 6,000 events generated with a $2.5\times$ oversample of defect-prone files, reported PRE_ONLY Precision@1 of 0.9398 and FUSED_ML Precision@1 of 0.9443—a fused-ML advantage of +0.45 pp. On the present real-defect LOPO corpus, the ordering reverses: PRE_ONLY at 0.7770 outperforms FUSED_ML at 0.7475 by -2.95 pp for the fusion method. This reversal is the single most important scientific update the rebuild produces and we take it at face value rather than reframe it.

Two mechanisms plausibly account for the reversal.

- **LOPO training regime.** The synthetic-data evaluation used an 80/20 event-stratified split within a fixed corpus, so FUSED_ML’s training set carried in-project signal that was absent from PRE_ONLY’s cross-project training set. Under LOPO, FUSED_ML must generalize across projects it has never seen, and the additional 10 failure-side features do not compensate for the loss of in-project familiarity. The 17-feature model over-fits per-project idiosyncrasies that do not transfer.
- **Absence of an inflating sampling bias.** The synthetic corpus over-sampled defect-prone files at $2.5\times$, producing an implicated-set defect rate of 64.2%. The real corpus’s implicated-set defect rate is

23.1%. The higher-defect-density synthetic regime rewarded any classifier that could distinguish among defect-prone candidates; the natural real-defect regime rewards a classifier that is well-calibrated across a much broader range of file characteristics, which is what the cross-project pre-execution predictor delivers.

Neither mechanism is a defect in the synthetic study’s internal validity—both were disclosed—but together they explain why the ranking-quality claim did not survive contact with real data.

6.2. Which feature groups hurt the fused-ML model?

The 17 features that FUSED_ML sees decompose into three groups: (i) seven cross-project process-metric features that also drive PRE_ONLY; (ii) the pre-execution score itself as an input; and (iii) nine failure-side features (five diff-family Bernoullis, one flake-rate, one locator-heal, one exception-type index, two HTTP-status one-hots). Group (i) is by construction identical to the PRE_ONLY feature set, and group (ii) simply forwards the pre-execution model’s own probability estimate. All of the fusion model’s extra capacity therefore has to be earned by group (iii). On real Defects4J and BugsInPy events, however, group (iii) is largely uninformative in ways that the synthetic corpus obscured. The dominant failing-test exception type across the corpus is a generic `AssertionError` or `ComparisonFailure` (Defects4J) or an equally generic `AssertionError` or `TypeError` (BugsInPy); the exception-type index has only three or four effective values and low mutual information with the true-defect label. The HTTP-status one-hots are structurally near-zero: only a handful of BugsInPy web-framework projects (`fastapi`, `sanic`, `tornado`, `httpie`) produce non-null HTTP statuses at all, so on 90%+ of events these features contribute pure noise. The diff-family Bernoullis `app_changed` / `validator_changed` / `schema_changed` / `fixture_changed` depend on a per-file tag map that was hand-curated on the synthetic corpus and applies imperfectly to real Java/Python projects, where the same source file may serve mixed roles. Historical flake rate is a fair signal but is dominated by a handful of chronically flaky files rather than being broadly discriminative. Under LOPO training the gradient booster does not have the sample budget to prune these weak signals correctly, and it fits per-project idiosyncrasies that do not transfer. A confirmatory ablation, retraining FUSED_ML on group (i) and group (ii) only (i.e. dropping all nine failure-side features) while keeping the same 150-estimator gradient-boosting configuration as full FUSED_ML, recovers Precision@1 of **0.7627**, an

improvement of 1.52 pp over the full-feature FUSED_ML (0.7475) but still 1.43 pp below PRE_ONLY (0.7770). The failure-side features are not merely uninformative on real data: they are actively hurting the model, accounting for roughly half of the FUSED_ML deficit. The remaining half is attributable to the higher-capacity ensemble (150 vs. 100 estimators) over-fitting the smaller per-project training folds. Neither factor is fatal in isolation; together they explain the full reversal cleanly. This decomposition is the strongest single piece of mechanistic support for the paper’s reversal claim and is directly attributable to the shift from synthetic to real corpora.

6.3. Calibration is the operationally meaningful finding

The synthetic study emphasised calibration as the fused-ML method’s principal contribution, reporting a $6.0\times$ ECE reduction over PRE_ONLY. On real data the ordering flips: PRE_ONLY is the best-calibrated method (ECE 0.0209), FUSED_ML is second (0.0326), FUSED_RULE is third (0.0626), and only the flat prior is uncalibrated. The direction of the update is that cross-project pre-execution defect priors on real corpora are *already extraordinarily well-calibrated*—so well that additional feature fusion cannot improve the number and mildly degrades it. This is a positive finding in a specific operational sense: an organisation that already deploys a cross-project file-level defect predictor can use its output probabilities directly as calibrated triage priors, without any post-hoc rescaling, ML fusion, or rule engineering.

6.4. What the fused methods are actually good for

FUSED_RULE’s 0.30-pp Precision@1 lead is real but small. Its ECE (0.0626) is worse than either of the classifier-based methods. The rule’s operational value is therefore not in either metric alone but in its transparency: the coefficients on `app_changed`, `validator_changed`, and `flake_rate` are inspectable by developers and can be adjusted per-team without retraining. FUSED_ML, by contrast, is worse than the baseline on ranking and mildly worse on calibration, and does not recover its cost as an operational tool on this corpus.

6.5. Comparison to the fault-localization literature at large

Our headline Precision@1 range of 0.75–0.78 is not directly comparable to line-level or method-level fault-localization numbers reported for Ochiai, DeepFL [12], FLUCCS [13], or LLM-based localizers [8, 9, 10] because those techniques address a different granularity (line or statement) and a different

input signal (test-coverage spectra or LLM prompting). The correct comparison for our setting is against the failure-only baseline, which represents today’s practice, and against the pre-only baseline, which represents the deployable defect-predictor-as-triage-aid alternative. On both comparisons the calibration finding is the one that generalises.

7. Threats to Validity

We enumerate ten threats to validity and describe what we did (or did not) do to mitigate each.

T1: Chart project skipped. One of the 17 Defects4J projects (Chart) did not check out reliably on our infrastructure at Defects4J v2.0.1, and its 26 bugs are excluded from the 824-bug Java total. The overall Defects4J bug count reported by Just et al. [5] is 835. The exclusion is a corpus-completeness issue rather than a methodology issue and is unlikely to change the direction of any headline finding, but it does mean the paper does not cover the full public Defects4J v2.0.1.

T2: Distractor sampling method. The implicated set for each event includes the fix-commit files plus distractor files sampled from other bugs’ fix commits within the same project. This is a synthetic-adjacent design choice and it deserves an explicit treatment. It sits between two more extreme alternatives. On one end, drawing distractors uniformly at random from the whole repository would produce an implicated set that bears no operational resemblance to what a CI system exposes on a real failure: most randomly sampled files are irrelevant to the failure by construction, would drive the empirical defect rate to a small fraction of a percent, and would make every ranking method look artificially perfect. On the other end, the ideal ground-truth implicated set would come from a coverage-instrumented run of the failing test—the set of files transitively touched by the failing execution—which is what a mature CI system would actually surface to a triaging developer. Producing that set requires re-running each of the 1,323 failing tests under a coverage tool, resolving language-specific instrumentation limits (JaCoCo for Defects4J’s older JDK targets, `coverage.py` for BugsInPy), and merging the coverage output back into an event-file table; we did not undertake this work at rebuild scope. The change-window distractor choice is a defensible middle ground: distractors are

actual production files that participated in real defect fixes in the same project’s recent history, and are therefore plausible candidates for a developer to consider under time pressure, without over-inflating the base rate. It is, however, weaker than a true coverage-based implicated set, and the direction of the residual bias is not guaranteed—distractors sampled from other bugs’ fix commits are systematically “defect-prone-ish” files, which may make our ranking task marginally harder than a coverage-based version would be. Future work should rerun the LOPO evaluation with coverage-instrumented failing tests to derive true implicated sets and confirm that the reversal-and-calibration findings survive the shift; we treat this as the single most important corpus-construction improvement remaining.

- T3: Cross-language transfer under LOPO.** The LOPO folds mix Java and Python projects in the training set for every held-out fold. On Java-heavy folds the Python-derived features may add noise; on Python-heavy folds the reverse. We did not evaluate a language-stratified LOPO variant.
- T4: Git-history feature staleness.** Process-metric features are drawn from each project’s git history up to the fix-commit date. For projects whose Defects4J or BugsInPy revision is many years old, the process metrics reflect a code base whose defect distribution may have shifted; we did not attempt to freeze features at a per-bug-relevant point in time.
- T5: LOPO leakage risks.** LOPO controls in-project leakage but not shared-authorship leakage across projects (a common committer’s habits may transfer) or shared-framework leakage (JacksonCore and JacksonDatabind, for example, share upstream design decisions). We did not exclude cross-project author or framework overlap.
- T6: Small BugsInPy repositories underrepresented.** Several BugsInPy projects contribute fewer than 10 events. Under LOPO these repos become high-variance leave-one-project folds whose per-project metrics carry wide confidence intervals; we report per-repository $p@1$ in Figure 4 but do not attempt per-repository significance testing at these sample sizes.

T7: External validity beyond OSS. Both corpora consist of open-source projects. Enterprise or closed-source code may follow different defect distributions, different code-review practices, and different test-suite compositions. The findings should be regarded as external-validity-bounded to OSS.

T8: Hyperparameter transfer from the synthetic study. Both the FUSED_RULE coefficients and the FUSED_ML hyperparameters were fixed at values chosen on the earlier synthetic study, without per-fold retuning. Re-tuning on a real-data validation split could recover part of the FUSED_ML gap but would introduce a per-fold overfitting concern; we chose the conservative option of holding hyperparameters constant.

T9: Metric choice. Precision@ k , Recall@ k , MRR, and 10-bin ECE reward top-of-list ranking and probability calibration respectively. Alternative task-oriented metrics—developer debugging time, cost-weighted attribution error—would require a user study.

T10: Ground-truth label completeness. Both Defects4J and BugsInPy label a single fix commit per bug, with the modified files taken as the true defects. If the actual defect spans a file not touched in the fix commit (a rare but non-zero phenomenon documented by Herzig et al. [22]) our metrics understate the true precision of every method uniformly.

8. Conclusion and Future Work

We rebuilt an empirical comparison of four post-execution defect attribution methods—PRE_ONLY, FAIL_ONLY, FUSED_RULE, and FUSED_ML—on 1,323 real defects drawn from Defects4J and BugsInPy across 33 projects, under a strict Leave-One-Project-Out cross-validation protocol. The headline result is a reversal of the fused-ML advantage previously reported on synthetic data: on real defects, the machine-learned fusion does not beat the cross-project pre-execution baseline. The hand-crafted rule-based fusion retains a marginal 0.30-pp Precision@1 lead. The calibration finding, however, is unambiguously positive: cross-project pre-execution defect priors on real corpora are extraordinarily well-calibrated (ECE 0.0209), and fusion does not improve on this.

Two operational implications follow. An organisation deploying a file-level defect predictor into CI triage can use its output probability directly as a calibrated per-file suspiciousness score; feature engineering the failure-side signal is unlikely to buy calibration and may degrade it. And an organisation that wants a small ranking boost can adopt a hand-crafted rule cascade whose coefficients remain inspectable; a machine-learned fusion is not justified by the real-data evidence.

Future work has three natural directions. First, replace the change-window distractor sampling with mined production-CI implicated sets (*e.g.*, via an industrial partnership) to close the last synthetic assumption in the corpus. Second, evaluate a language-stratified LOPO variant to isolate cross-language transfer effects. Third, extend the corpus to a third real-defect benchmark (*e.g.*, a CVEfixes [20] slice restricted to a single CWE family) to probe generalisation to security defects.

Acknowledgements

The Defects4J and BugsInPy maintainers made this study possible by curating and maintaining these open-source real-defect corpora.

CRedit authorship contribution statement

Vijay Prasad Javvadi: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing — original draft, Writing — review & editing.

Declaration of competing interest

The author declares no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability

A reproducibility bundle comprising the LOPO event corpus (`events_real.parquet`), the four attribution-method implementations, the per-event metrics CSV, and the calibration-figure generation scripts will be archived on Zenodo prior to acceptance and a DOI added to the camera-ready version. The source code is maintained in a public GitHub repository under the author’s account (`javvadivijayprasad/paperF_defect_attribution`); the repository link and Zenodo DOI will be inserted at the proof stage. At the time of submission the DOI has not yet been minted; readers may contact the author to obtain a pre-archival snapshot for peer review.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Anthropic Claude Sonnet in order to assist with drafting, structural revision, and prose editing. All experimental infrastructure, code, data-ingestion pipelines, statistical analyses, calibration computations, and interpretation are the author’s own. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Author biography

Vijay Prasad Javvadi is an Independent Researcher based in Plainsboro, New Jersey, USA. He holds a B.Tech in Computer Science and Engineering and works on empirical software engineering, LLM-for-software-engineering methods, defect prediction, and test-automation tooling. He is the author of a portfolio of published open-source test-automation packages on npm (including `bdd2pw`, `sel2pw`, `pw-emit`, `usagecontract`, `pw-self-heal`, and `synthdata`), and his tool `bdd2pw` has been published in Elsevier’s SoftwareX. His current research focuses on calibrated defect attribution, cross-project generalisation of prediction models, and reproducible empirical evaluation on real-defect corpora.

References

- [1] N. Nagappan, T. Ball, Use of relative code churn measures to predict system defect density, in: Proceedings of the 27th International Conference on Software Engineering (ICSE), 2005, pp. 284–292.

- [2] R. Moser, W. Pedrycz, G. Succi, A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction, in: Proceedings of the 30th International Conference on Software Engineering (ICSE), 2008, pp. 181–190.
- [3] F. Rahman, P. Devanbu, How, and why, process metrics are better, in: Proceedings of the 35th International Conference on Software Engineering (ICSE), 2013, pp. 432–441.
- [4] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, N. Ubayashi, A large-scale empirical study of just-in-time quality assurance, *IEEE Transactions on Software Engineering* 39 (6) (2013) 757–773.
- [5] R. Just, D. Jalali, M. D. Ernst, Defects4J: A database of existing faults to enable controlled testing studies for Java programs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA), ACM, 2014, pp. 437–440. [doi:10.1145/2610384.2628055](https://doi.org/10.1145/2610384.2628055).
- [6] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, E. L. Ouh, BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies, in: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2020, pp. 1556–1560. [doi:10.1145/3368089.3417943](https://doi.org/10.1145/3368089.3417943).
- [7] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.
- [8] A. Z. H. Yang, C. Le Goues, R. Martins, V. J. Hellendoorn, Large language models for test-free fault localization, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24), ACM, 2024, pp. 17:1–17:12. [doi:10.1145/3597503.3623342](https://doi.org/10.1145/3597503.3623342).
- [9] S. Kang, G. An, S. Yoo, A quantitative and qualitative evaluation of LLM-Based explainable fault localization, *Proceedings of the ACM on Software Engineering (FSE)* 1 (FSE) (2024). [doi:10.1145/3660771](https://doi.org/10.1145/3660771).

- [10] M. Rezaalipour, C. A. Furia, An empirical study of fault localization in Python programs, *Empirical Software Engineering* 29 (4) (2024) 92. [doi:10.1007/s10664-024-10475-3](https://doi.org/10.1007/s10664-024-10475-3).
- [11] R. Abreu, P. Zoetewij, A. J. C. van Gemund, Spectrum-based multiple fault localization, in: *Proceedings of the 24th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2009, pp. 88–99.
- [12] X. Li, W. Li, Y. Zhang, L. Zhang, DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization, in: *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, ACM, 2019, pp. 169–180.
- [13] J. Sohn, S. Yoo, FLUCCS: Using code and change metrics to improve fault localization, in: *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, ACM, 2017, pp. 273–283.
- [14] X. Zhou, D. Han, D. Lo, Bridging expert knowledge with deep learning techniques for just-in-time defect prediction, *Empirical Software Engineering* 30 (2) (2025) 37. [doi:10.1007/s10664-024-10591-0](https://doi.org/10.1007/s10664-024-10591-0).
- [15] X. Shahini, A. Metzger, K. Pohl, An empirical study on just-in-time conformal defect prediction, in: *Proceedings of the 21st International Conference on Mining Software Repositories (MSR '24)*, ACM, 2024, pp. 88–99. [doi:10.1145/3643991.3644928](https://doi.org/10.1145/3643991.3644928).
- [16] Z. Li, Q. Du, H. Zhang, Y. Wang, X. Chen, An empirical study of data sampling techniques for just-in-time software defect prediction, *Automated Software Engineering* 31 (2) (2024) 56. [doi:10.1007/s10515-024-00455-8](https://doi.org/10.1007/s10515-024-00455-8).
- [17] M. Chouchen, A. Ouni, G. K. Rajbahadur, M. Kessentini, M. W. Mkaouer, Less is more: Balancing models performance and complexity for software defects prediction, *Empirical Software Engineering* 32 (1) (2026) 5. [doi:10.1007/s10664-026-10913-4](https://doi.org/10.1007/s10664-026-10913-4).
- [18] Y. Hong, C. Tantithamthavorn, J. Pasuksmit, P. Thongtanunam, A. Friedman, X. Zhao, A. Krasikov, Practitioners' challenges and perceptions of CI build failure predictions at Atlassian, in: *Companion*

- Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24), ACM, 2024, pp. 370–381. [doi:10.1145/3663529.3663856](https://doi.org/10.1145/3663529.3663856).
- [19] A. Niculescu-Mizil, R. Caruana, Predicting good probabilities with supervised learning, in: Proceedings of the 22nd International Conference on Machine Learning (ICML), ACM, 2005, pp. 625–632.
 - [20] J. Śliwerski, T. Zimmermann, A. Zeller, When do changes induce fixes?, in: Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR), 2005, pp. 1–5.
 - [21] C. Bird, A. Bachmann, E. Aune, J. Duffy, A. Bernstein, V. Filkov, P. Devanbu, Fair and balanced? bias in bug-fix datasets, in: Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE), 2009, pp. 121–130.
 - [22] K. Herzig, S. Just, A. Zeller, It’s not a bug, it’s a feature: How misclassification impacts bug prediction, in: Proceedings of the 35th International Conference on Software Engineering (ICSE), IEEE, 2013, pp. 392–401.
 - [23] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, R. Oliveto, Evaluating SZZ implementations through a developer-informed oracle, Proceedings of the 43rd International Conference on Software Engineering (ICSE) (2021) 436–447.
 - [24] P. Rani, F. Petruccio, A. Bacchelli, On refining the SZZ algorithm with bug discussion data, Empirical Software Engineering 29 (5) (2024) 115. [doi:10.1007/s10664-024-10511-2](https://doi.org/10.1007/s10664-024-10511-2).
 - [25] S. Perez-Rosero, R. Dyer, S. W. Flint, S. McIntosh, W. Srisa-An, WIA-SZZ: Work item aware SZZ, Empirical Software Engineering 30 (3) (2025) 75. [doi:10.1007/s10664-025-10616-2](https://doi.org/10.1007/s10664-025-10616-2).
 - [26] O. Parry, G. M. Kapfhammer, M. Hilton, P. McMinn, Systemic flakiness: An empirical analysis of co-occurring flaky test failures, in: Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25), ACM, 2025, pp. 476–487. [doi:10.1145/3756681.3756945](https://doi.org/10.1145/3756681.3756945).

- [27] B. Vasilescu, Y. Yu, H. Wang, P. Devanbu, V. Filkov, Quality and productivity outcomes relating to continuous integration in GitHub, in: Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2018, pp. 805–816.
- [28] T. Zimmermann, N. Nagappan, H. Gall, E. Giger, B. Murphy, Cross-project defect prediction: A large scale experiment on data vs. domain vs. process, in: Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE), 2009, pp. 91–100.
- [29] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, K. Matsumoto, An empirical comparison of model validation techniques for defect prediction models, *IEEE Transactions on Software Engineering* 43 (1) (2017) 1–18.